

can generate fake output in the absence of actual data for the user query. Since the data is retrieved from existing sources and the prompt is augmented, hallucinations can be avoided. Open source frameworks like LangChain and LlamaIndex have made building RAG-based applications easier. This has led to enterprises adopting this approach for building enterprise search and Q&A conversational applications that leverage private data.

Implementing a chatbot for enterprise data search

Here's a step-by-step implementation of the enterprise search application built using Python and Google's GCP Vertex AI platform. The LangChain framework is used to simplify building the RAG system, which can be classified into two layers – a data ingestion layer and a data serving layer.

Data ingestion layer

The responsibility of the data ingestion layer is to load the data and make it available in a format that can be used by the data serving layer.

Loading the documents from GCS: The first step is to make the private enterprise data available to the application.

In this sample application, we use Google Cloud Storage (GCS) bucket as the data store. Two financial documents in PDF format – one related to Uber and one of Alphabet (Google) -- are used for the demo. LangChain provides multiple utility classes to connect and load documents from various data sources. Here, GCSDirectoryLoader from langchain_google_community is used to load the documents from the GCS bucket.

```
loader = GCSDirectoryLoader(project_name=PROJECT_ID,
bucket="rag-langchain-demo")
documents = loader.load()
```

Splitting documents into chunks: Documents are usually split into chunks due to factors like efficiency and cost. Various chunking strategies are possible – character count based, sentence based, page level chunking, etc. Here we will use character chunking with a chunk size of 10000 characters and overlap of 100.

```
text_splitter = RecursiveCharacterTextSplitter(chunk_
size=10000, chunk_overlap=100)
doc_chunks = text_splitter.split_documents(documents)
```

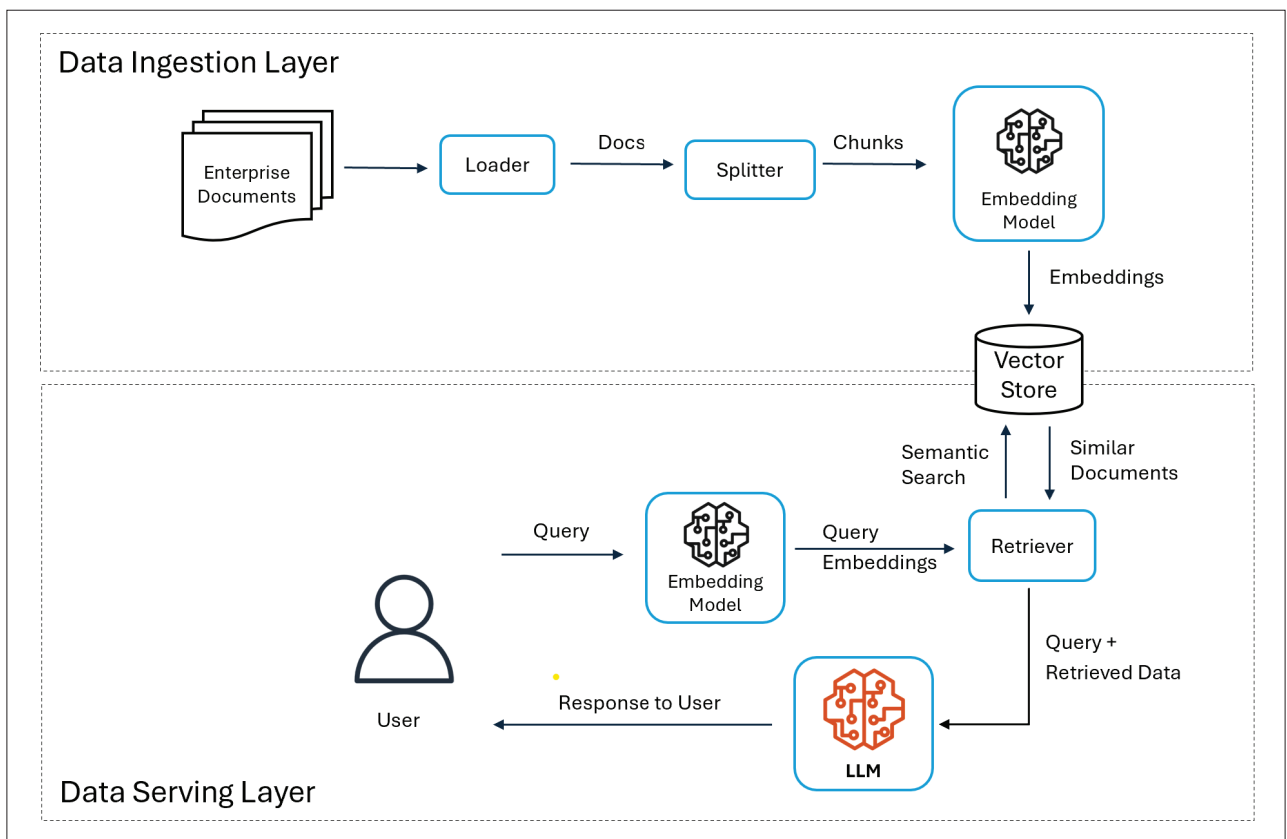


Figure 1: GenAI chatbot for enterprise data search